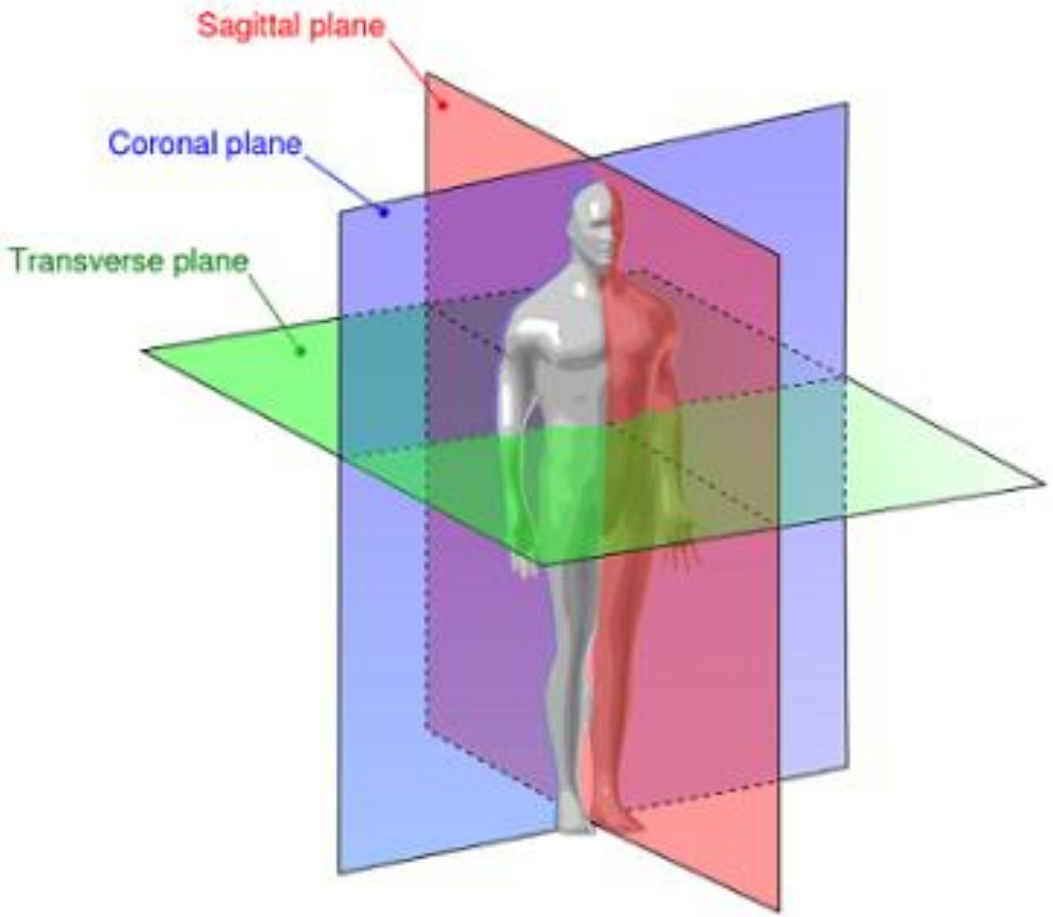
人工智能与医学数据

- 医学影像基础概念
- 医学影像文件格式介绍

医学影像基础概念



切面：

- 横断面（axial plane）

与地面平行，分离头部（Superior）与脚部（Inferior）

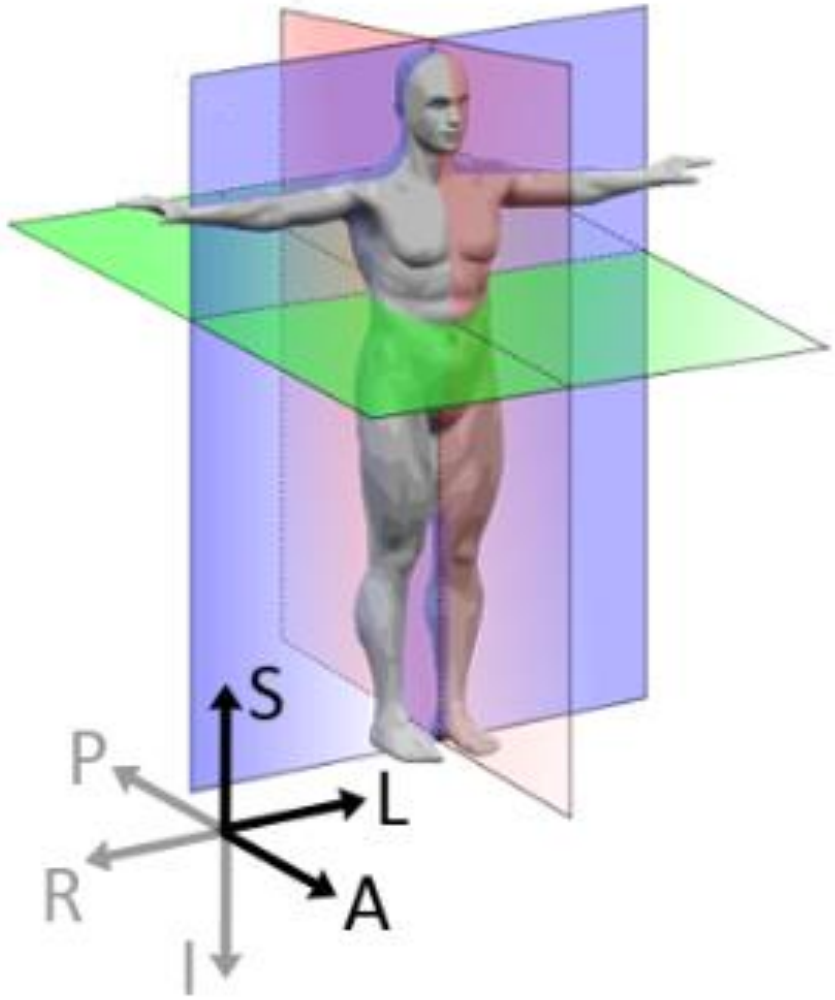
- 冠状面（coronal plane）

与地面垂直，分离人体的前（Anterior）后（Posterior）

- 矢状面（sagittal plane）

与地面垂直，分离人体的左（Left）右（Right）

医学影像基础概念



方向

以站立的病人自身为参考, 如: 右侧表示病人的右侧, 如下图所示. **病人的右侧通常在图像显示在左侧.** 六个方向:

- 左 (**L**eft)
- 右 (**R**ight)
- 前 (**A**nterior)
- 后 (**P**osterior)
- 上 (**S**uperior)
- 下 (**I**nferior)

注：使用 up, down, front, back 是不准确的, 因为病人在站立和躺下时这几个概念会混淆。

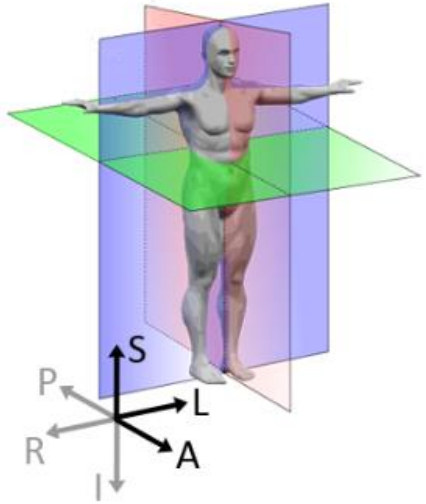
医学影像基础概念

世界坐标系

- 图像处理中通常运用三种坐标体系：世界坐标系，解剖坐标系（a.k.a. 病人坐标系）和图像坐标系。
- 世界坐标系也称为测量坐标系，是一个三维直角坐标系，以其为基准可以描述相机和待测物体的空间位置。世界坐标系的位置可以根据实际情况自由确定。
- 常见物理坐标系类型：
  - 直角坐标系 (笛卡尔坐标系)：最常用。
  - 柱坐标系 (Cylindrical)：用于研究圆柱对称问题。由径向距离
  - 球坐标系 (Spherical)：用于研究球面或中心对称问题。

医学影像基础概念

解剖坐标系



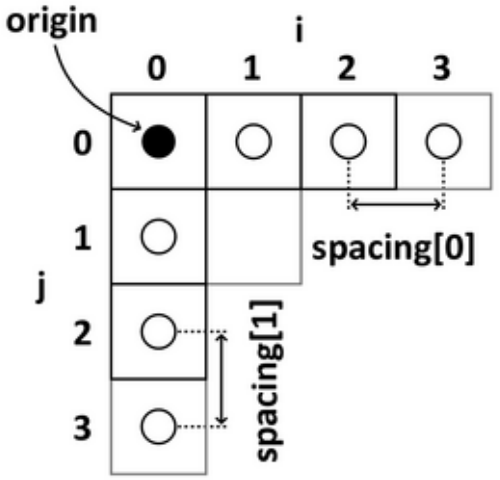
- 解剖坐标系是一个基于解剖结构的坐标系，用于描述人体或动物的解剖结构。该坐标系通常由解剖学结构的方向和位置表示，如左右、前后、上下。解剖学坐标系的优点是可以更直观地描述解剖结构的方向和位置，但它不适合用于图像处理和分析。该坐标系是**建立在特定解剖结构上的坐标系，用于描述解剖结构在人体中的位置和姿态。**解剖学坐标系能够提供与人体解剖结构相关的信息，但对于不同的病人或不同的解剖学结构，坐标系会有所不同，因此需要进行针对性的建模。
- 解剖坐标系，也称为病人坐标系，是一个连续的三维空间, 图像从该空间中采样. 解剖坐标系使用上述的六个方向作为坐标系的正负轴,
- 确定三个轴的**正方向**通常有以下两种:
  - **LPS(Left, Posterior, Superior):** MHD图像(meta image), ITK工具包, ITK-Snap软件(该软件中写的是RAI)使用
  - **RAS(Right, Anterior, Superior):** Nifti图像和3D Slicer软件使用

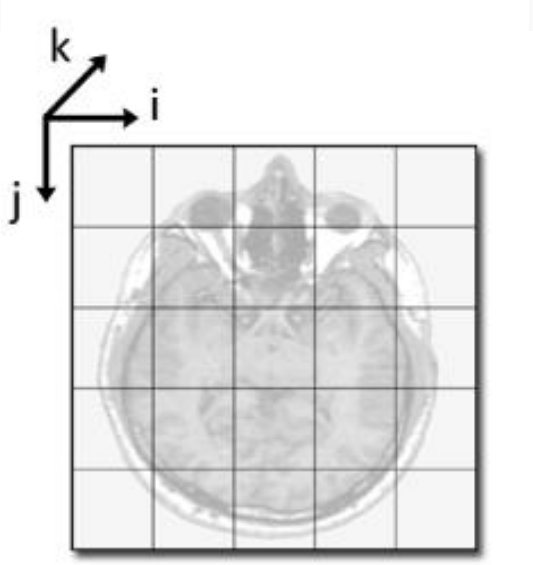
  以上两种均为右手坐标系.
- 注：人们并不总是用三个连续的字母表示正方向, 有时候也表示出发(from)的方向(也就是负方向), 此时上面的LPS会被写成RAI, 对应的到达(to)方向才是LPS. 比如ITK-Snap软件中Tools->Image Information->Orientation就写的是RAI, 表达的意思是 from RAI --> to LPS , 同样的使用 ITK 导出的MHD格式图像也是使用From模式.

医学影像基础概念

图像坐标系

- 图像坐标系是医学图像中最常用的坐标系，用于描述像素在图像中的位置。
- 该坐标系通常是一个二维网格坐标系，由行和列的像素值组成。
- 图像坐标系的优点是适合进行图像处理和分析，但缺点是不直观，需要通过图像转换技术将其与其他坐标系进行对应。
- 图像坐标系统中不仅包含了体素值(三维中的像素一般称为体素 voxel), 还包含采样原点(origin), 采样间距(spacing)等信息。
- 坐标转换。





医学影像文件格式介绍

Nifti 文件格式 (*.nii, *.nii.gz)

1. 什么是NIFTI?

- NIFTI（Neuroimaging Informatics Technology Initiative，**神经影像信息学技术倡议**）文件格式，是目前各大神经影像分析工具普遍兼容的体素水平的数据格式，也是在进行神经影像研究中**最常见**的一种数据格式。
- 它是一个三维数组或者四维**数组**，再套上一个**头部数据**。数组里包含的就是**图像体素值数据**本身，头部数据里包含**空间和体素信息。**
- NIFTI最初是用于神经成像的，但它也适用于一些其他的领域。标准NIfTI图像的**扩展名**是**.nii**，也可以写 为**.nii.gz**，总之，.nii格式和.nii.gz格式是一个东西，可以在文件名上直接修改来相互转换。
- Nifti 格式使用 RAS (Right, Anterior, Superior) 作为坐标系的正方向.

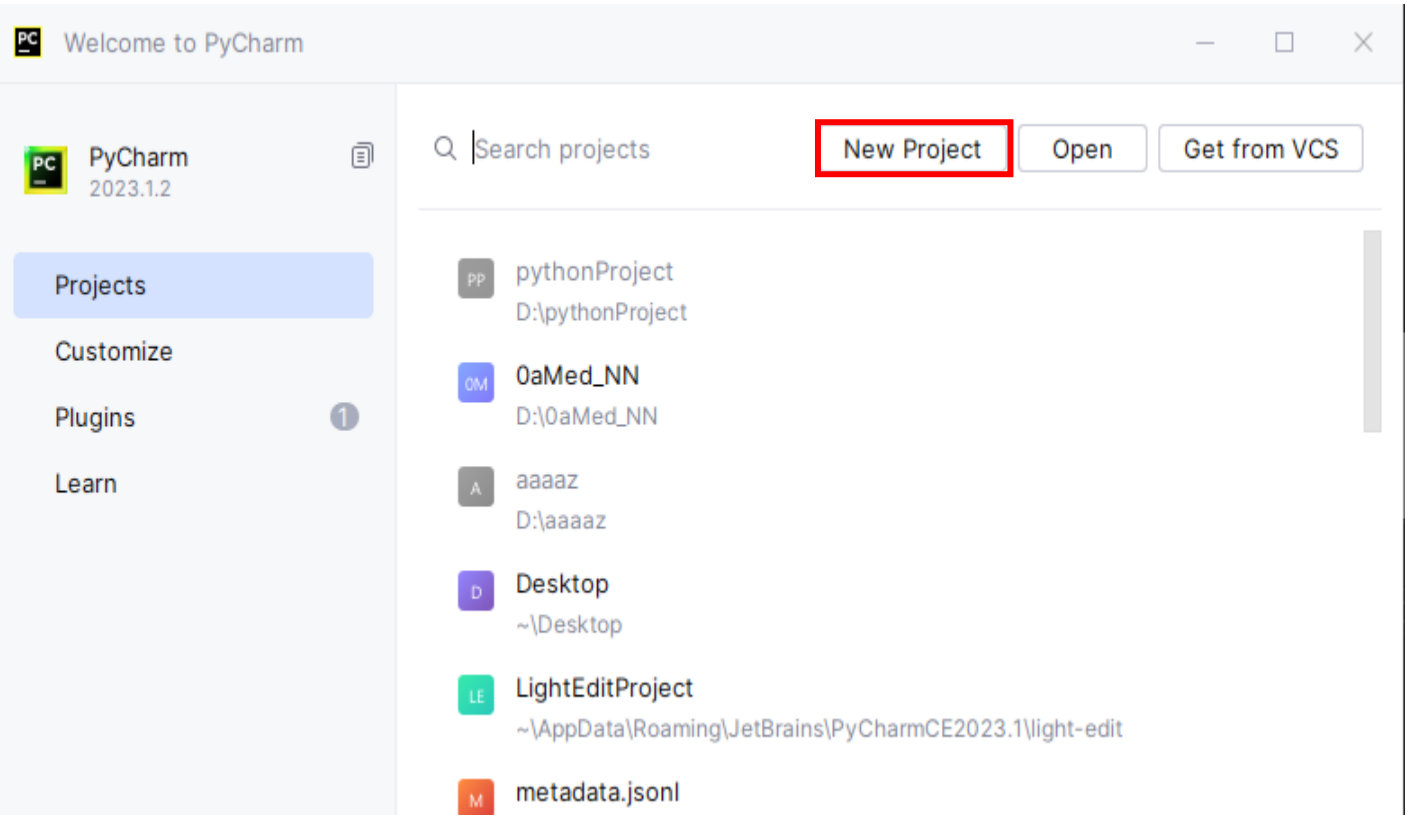
2. 头文件各字段内容与意义

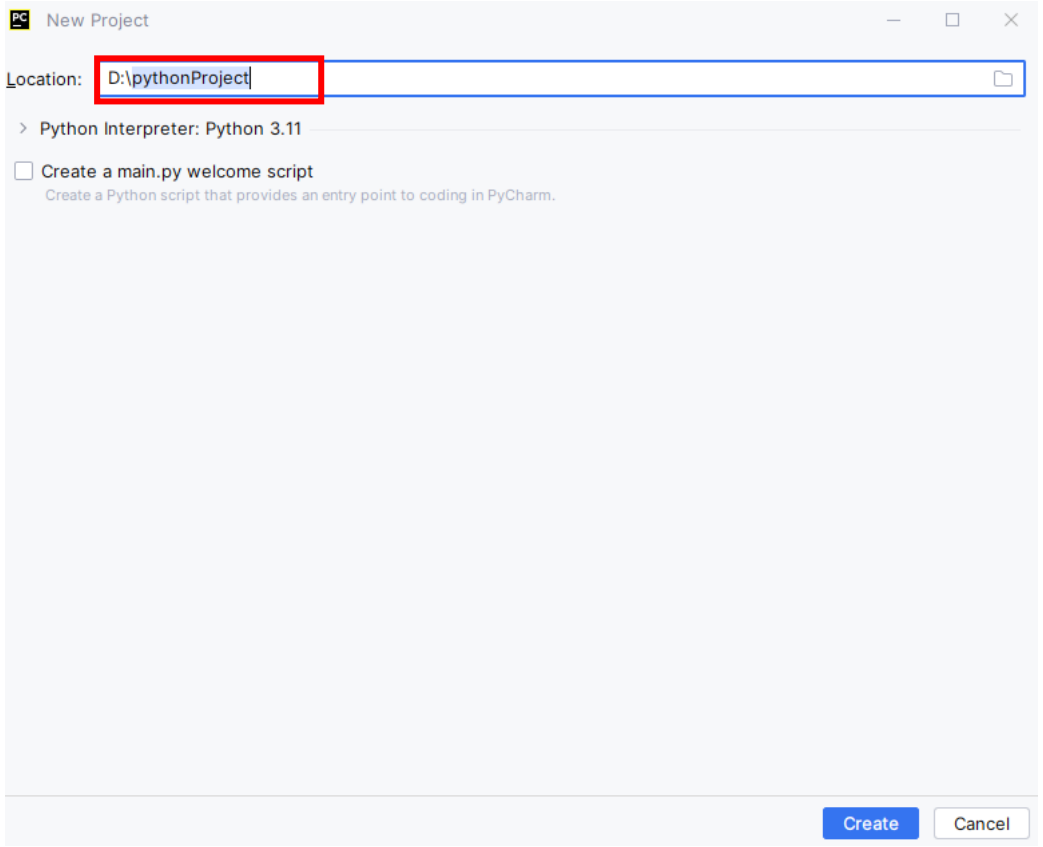
- int sizeof_hdr　　保存文件的头文件大小
- char dim_info　　存储着频率编码方向、相位编码方向和采集期间层选择方向
- short dim　　存储着图像的维度信息
- intent系列　　说明数据的目的
- slice信息　　存储磁共振成像采集的时间相关信息
- float pixdim　　体素维度
- ……

Nifti 文件格式 (*.nii, *.nii.gz)

3. Python读取NIFTI文件：使用nibabel库

- 打开PyCharm 并执行以下操作

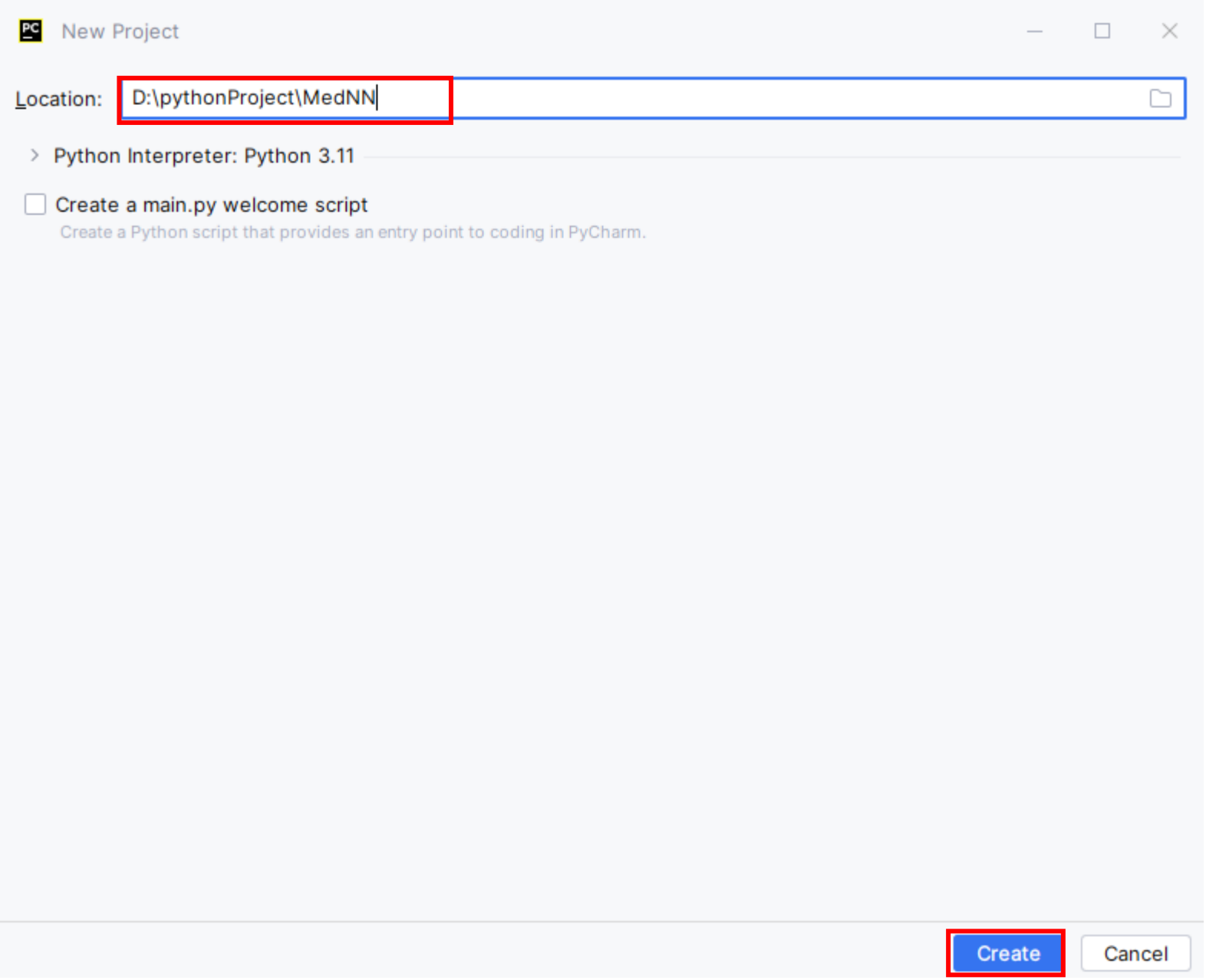




Nifti 文件格式 (*.nii, *.nii.gz)

Python读取NIFTI文件：使用nibabel库

- 打开PyCharm 并执行以下操作

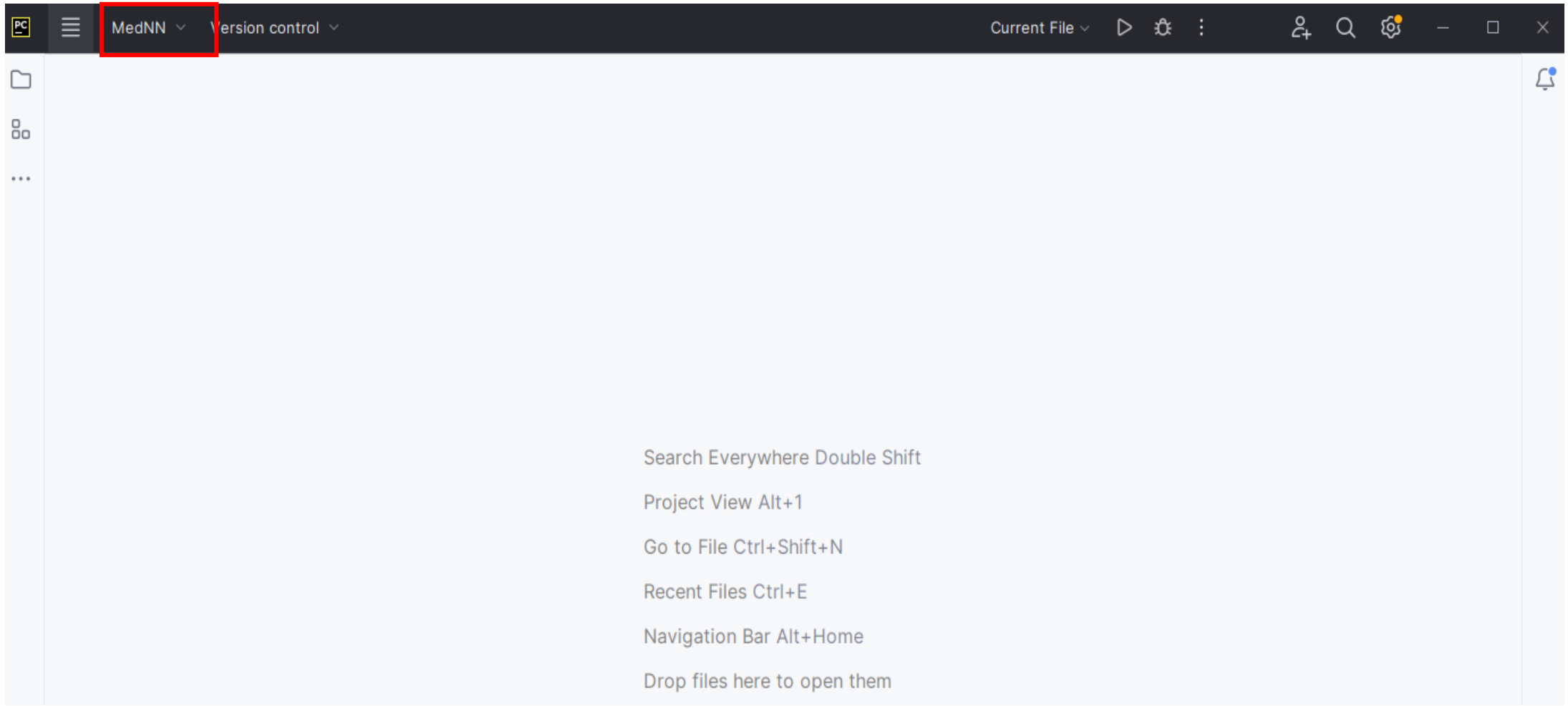


注：避免中文路径，防止报错

Nifti 文件格式 (*.nii, *.nii.gz)

Python读取NIFTI文件：使用nibabel库

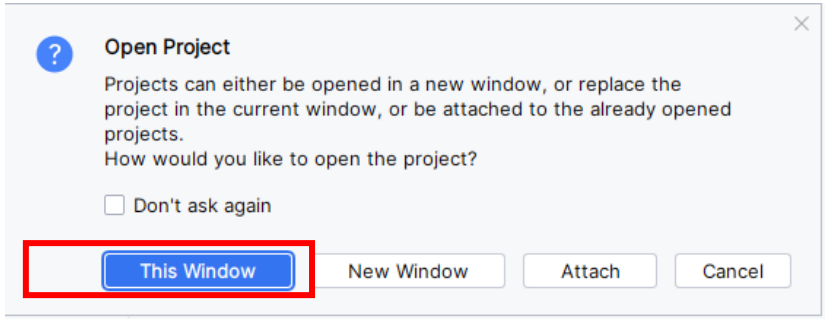
- 执行完以上操作，将会出现如下信息

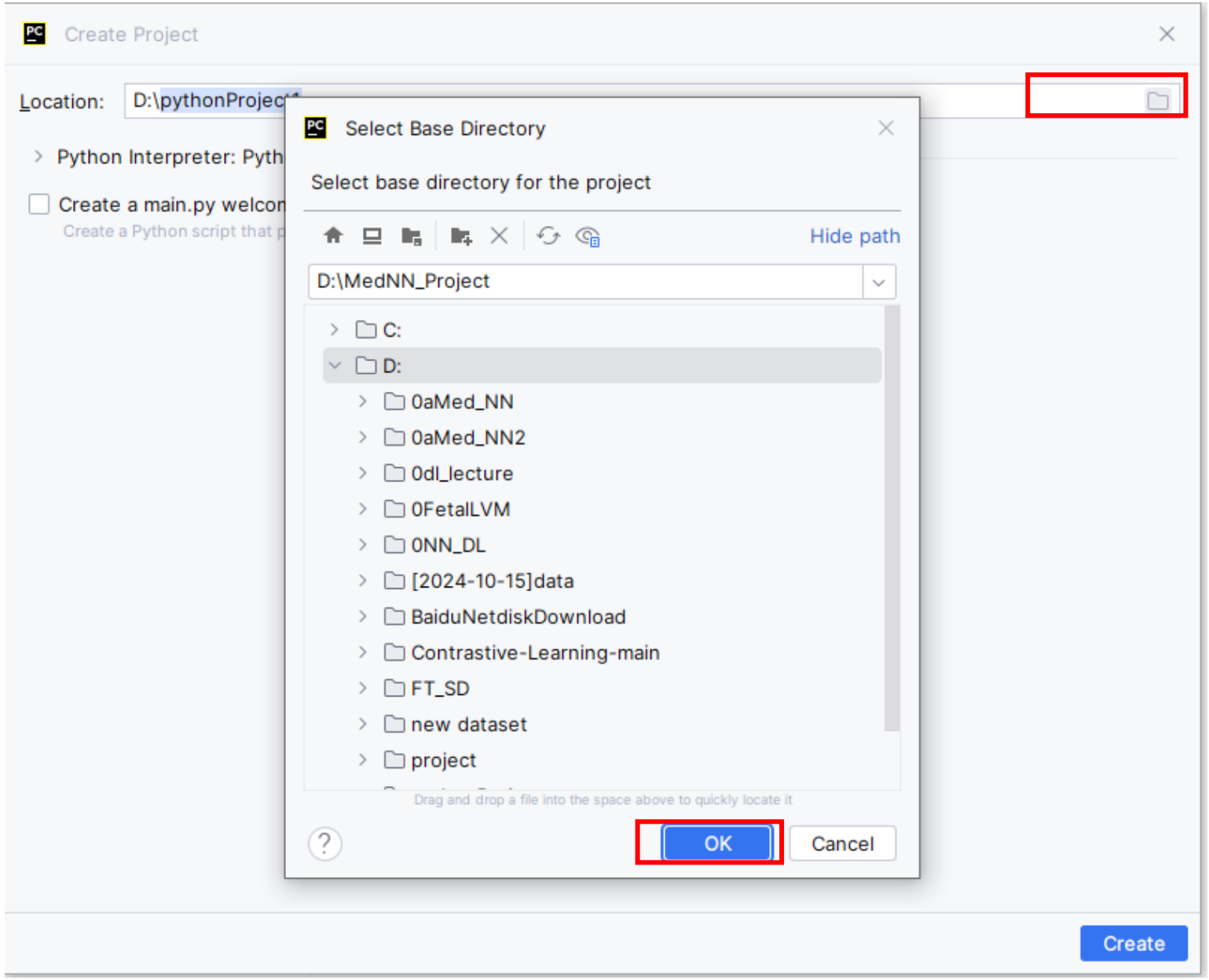


Nifti 文件格式 (*.nii, *.nii.gz)

Python读取NIFTI文件：使用nibabel库

- 或者打开PyCharm， **File >> New Project**



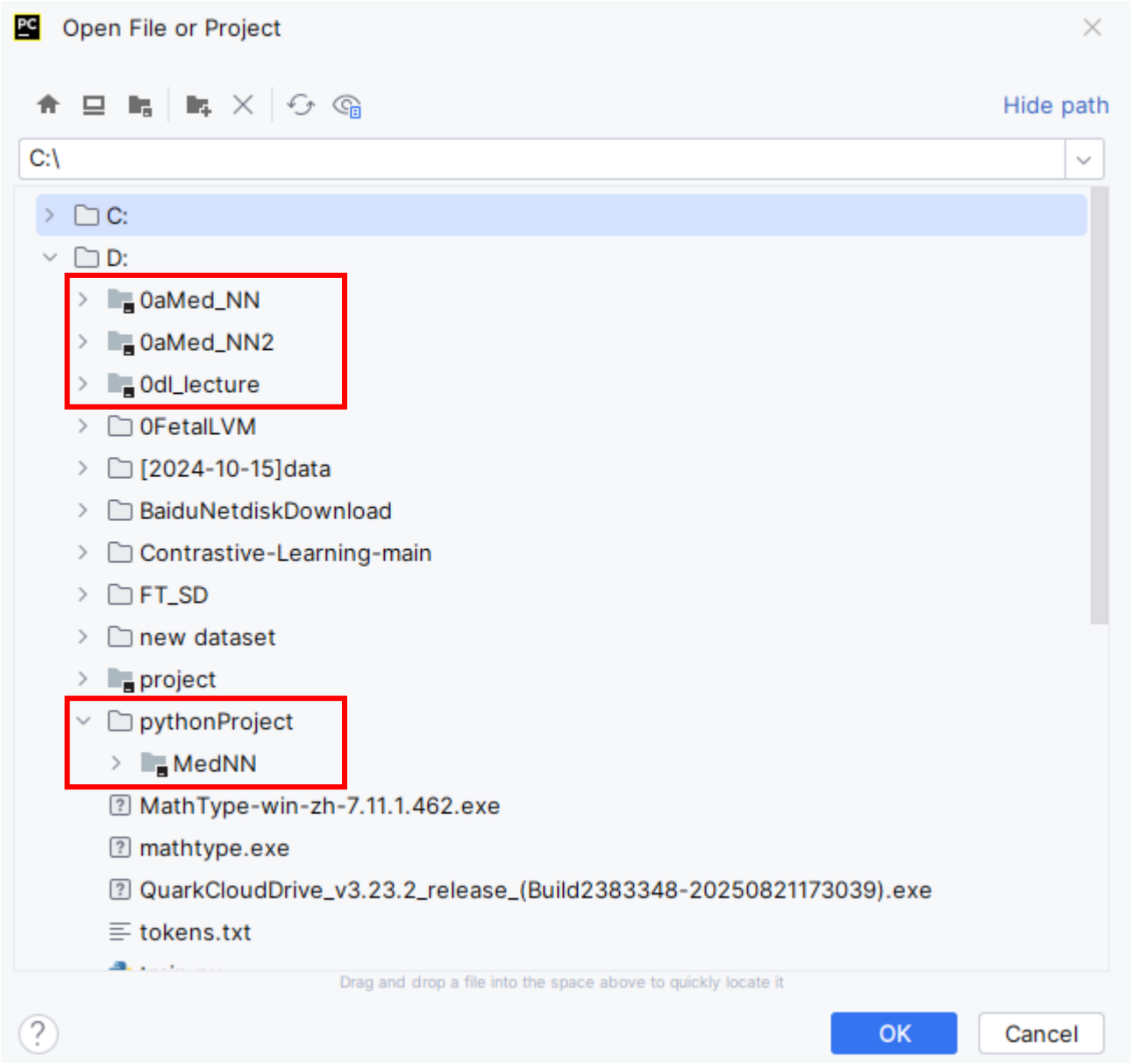


Nifti 文件格式 (*.nii, *.nii.gz)

Python读取NIFTI文件：使用nibabel库

打开其他project 的操作：

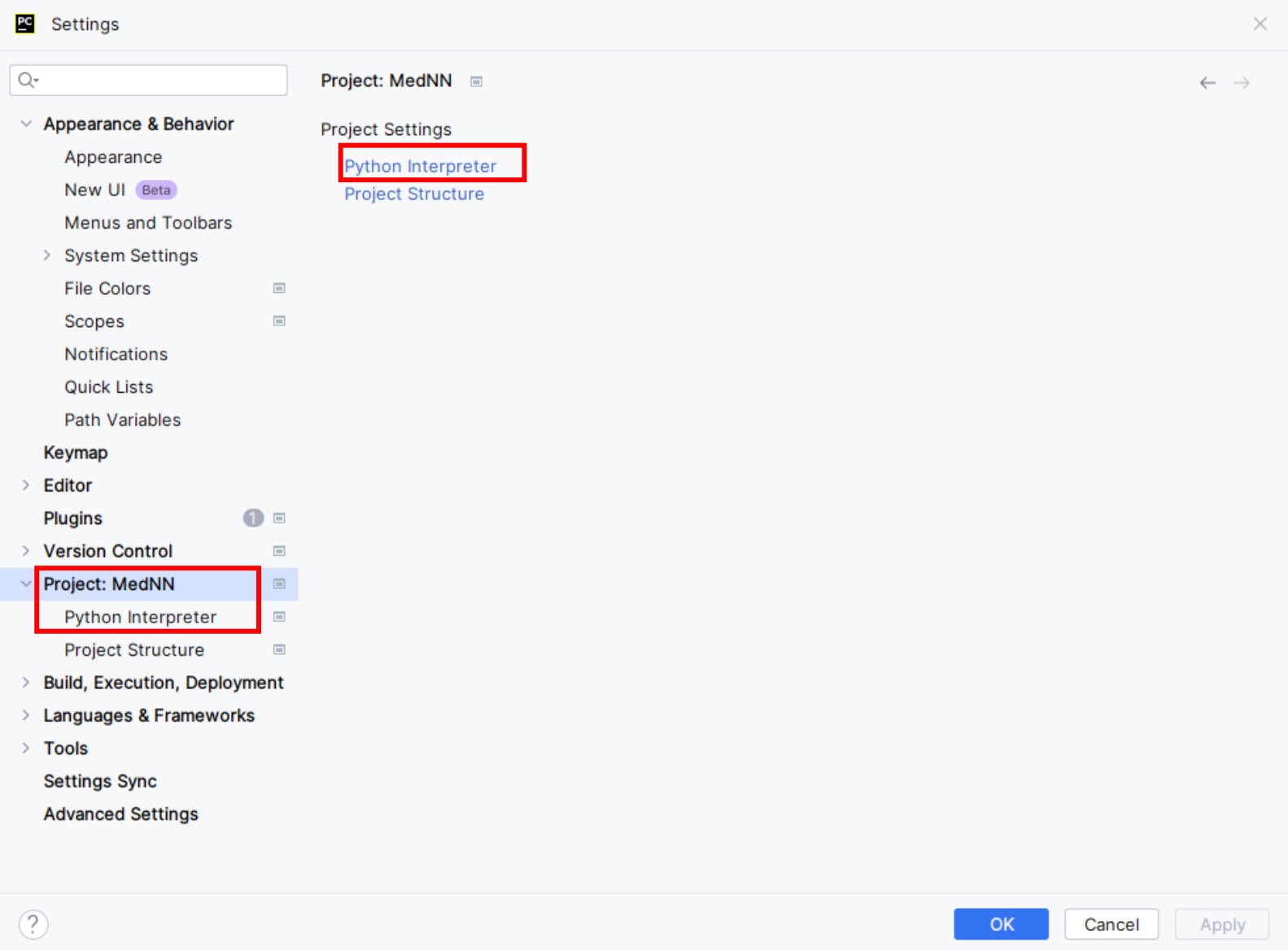
- 打开PyCharm， **File >> Open,** 将弹出对话框



Nifti 文件格式 (*.nii, *.nii.gz)

Python读取NIFTI文件：使用nibabel库

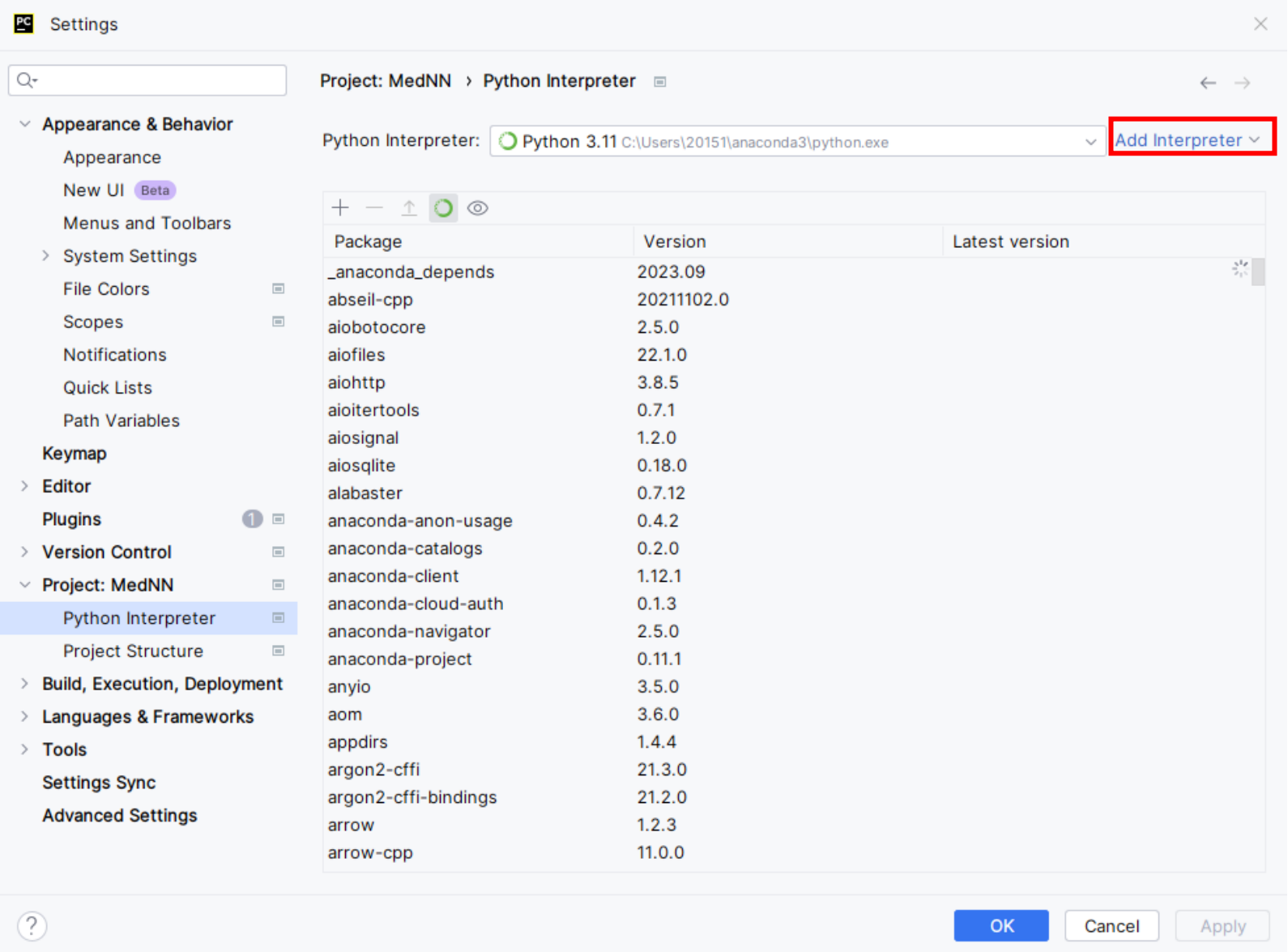
- 配置环境： 右上角点击齿轮，点击Settings
- 弹出的对话框中，依次执行一下操作



Nifti 文件格式 (*.nii, *.nii.gz)

Python读取NIFTI文件：使用nibabel库

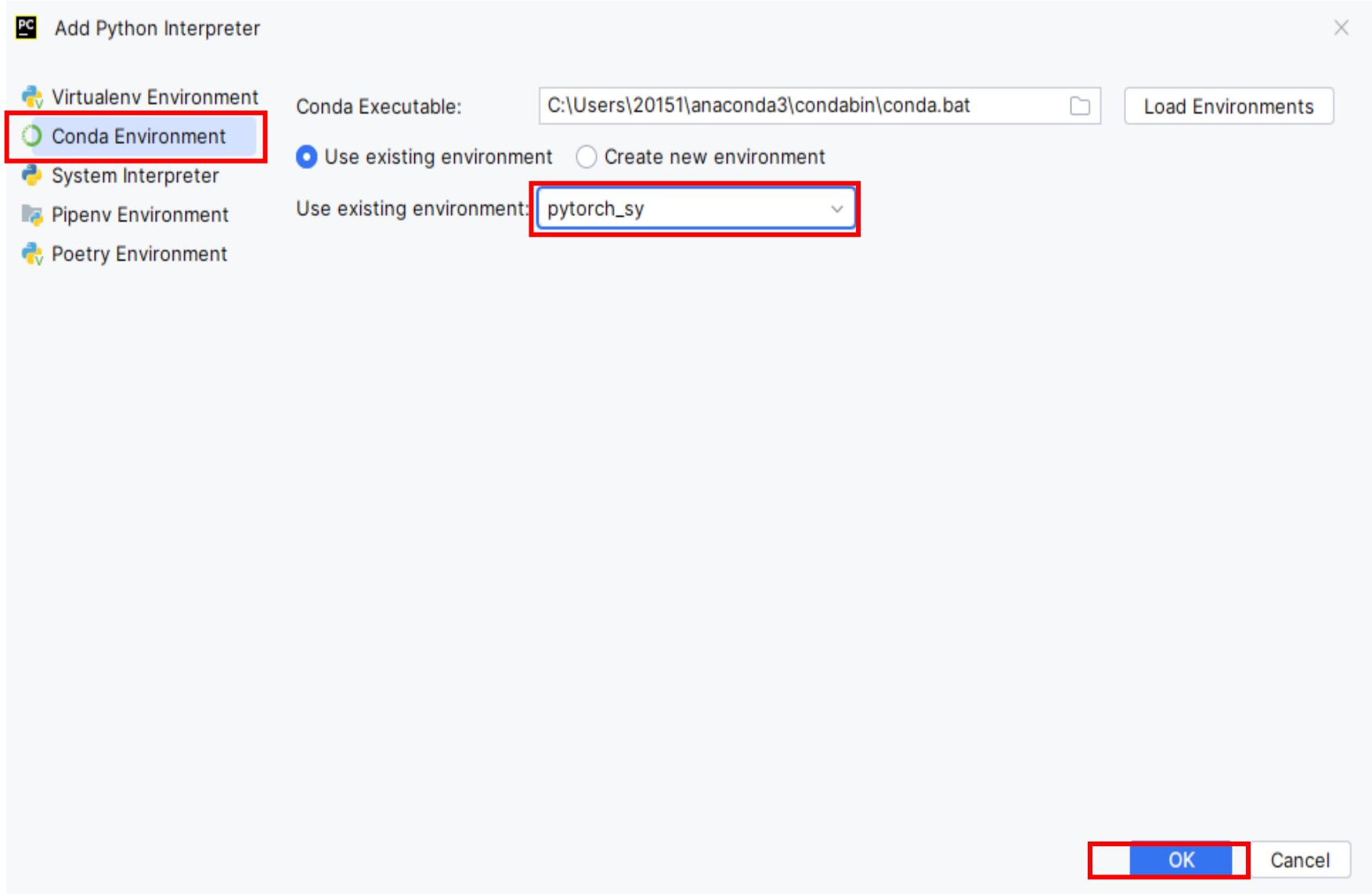
- 配置环境： 右上角点击齿轮，点击Settings
- 弹出的对话框中，依次执行一下操作



Nifti 文件格式 (*.nii, *.nii.gz)

Python读取NIFTI文件：使用nibabel库

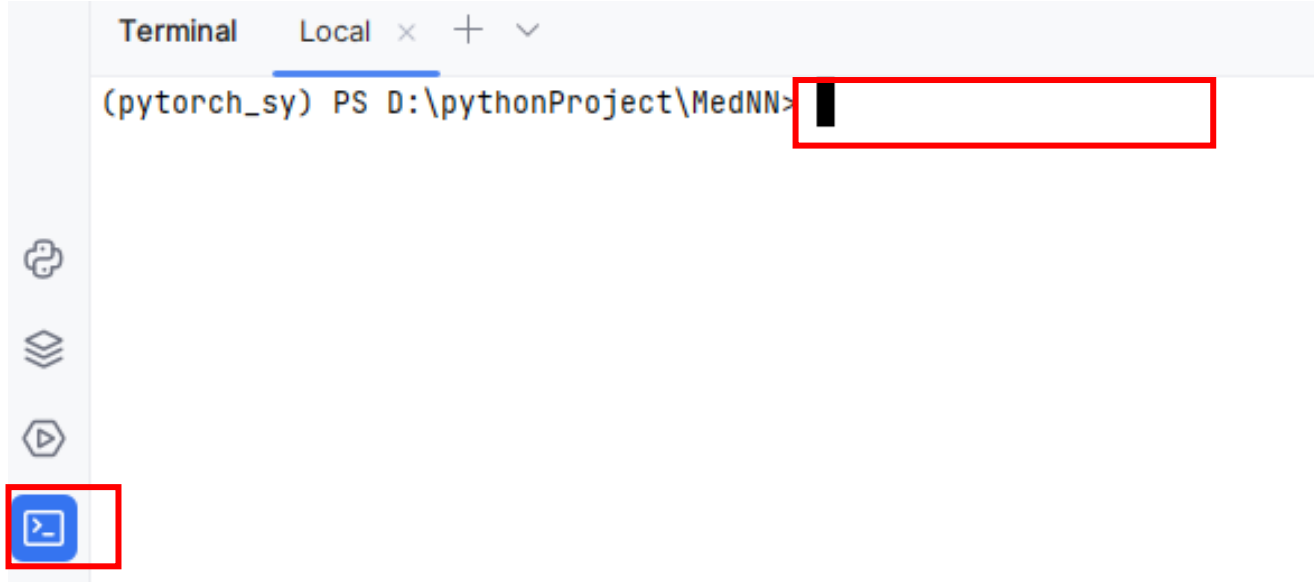
- 配置环境： 右上角点击齿轮，点击Settings
- 弹出的对话框中，依次执行一下操作



Nifti 文件格式 (*.nii, *.nii.gz)

Python读取NIFTI文件：使用nibabel库

- 安装nibabel库
  - 点击左下角的Terminal，输入
    pip install nibabel
  - 或者在anaconda prompt中激活虚拟环境，再输入命令




```
(pytorch_sy) D:\0aMed_NN>pip install nibabel
```

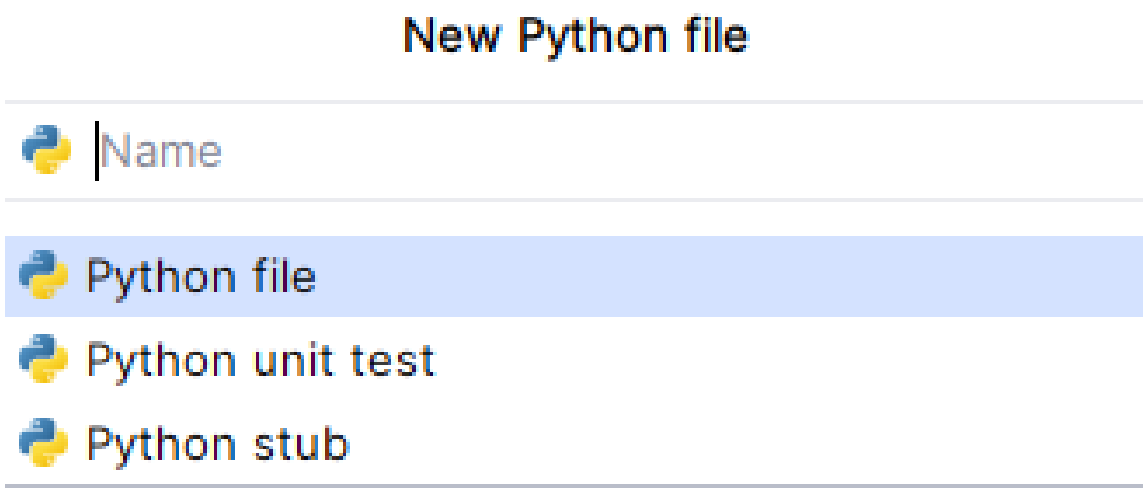
```
(pytorch_sy) D:\0aMed_NN>pip install nibabel
Looking in indexes: https://pypi.tuna.tsinghua.edu.cn/simple, http://pypi.douban.com/simple, http://mirrors.aliyun.com/pypi/simple, https://pypi.tuna.tsinghua.edu.cn/simple, http://pypi.mirrors.ustc.edu.cn/simple, http://pypi.hustunique.
com
WARNING: Retrying (Retry(total=4, connect=None, read=None, redirect=None, status=None)) after connection broken by 'ProtocolError('Connection aborted.', RemoteDisconnected('Remote end closed connection without response'))': /simple/nibab
el/
WARNING: Retrying (Retry(total=3, connect=None, read=None, redirect=None, status=None)) after connection broken by 'ProtocolError('Connection aborted.', RemoteDisconnected('Remote end closed connection without response'))': /simple/nibab
el/
WARNING: Retrying (Retry(total=2, connect=None, read=None, redirect=None, status=None)) after connection broken by 'ProtocolError('Connection aborted.', RemoteDisconnected('Remote end closed connection without response'))': /simple/nibab
el/
WARNING: Retrying (Retry(total=1, connect=None, read=None, redirect=None, status=None)) after connection broken by 'ProtocolError('Connection aborted.', RemoteDisconnected('Remote end closed connection without response'))': /simple/nibab
el/
WARNING: Retrying (Retry(total=0, connect=None, read=None, redirect=None, status=None)) after connection broken by 'ProtocolError('Connection aborted.', RemoteDisconnected('Remote end closed connection without response'))': /simple/nibab
el/
Collecting nibabel
  Downloading http://mirrors.ustc.edu.cn/pypi/packages/89/51/c97641cc2cd1b3b14cdec54b4e86fe03fc59753ecd13dc67544716fb7353/nibabel-4.0.2-py3-none-any.whl (3.3 MB)
     ---------------------------------------- 3.3/3.3 MB 9.3 MB/s eta 0:00:00
Requirement already satisfied: numpy>=1.17 in c:\users\20151\anaconda3\envs\pytorch_sy\lib\site-packages (from nibabel) (1.21.5)
Requirement already satisfied: packaging>=17.0 in c:\users\20151\anaconda3\envs\pytorch_sy\lib\site-packages (from nibabel) (23.2)
Requirement already satisfied: setuptools in c:\users\20151\anaconda3\envs\pytorch_sy\lib\site-packages (from nibabel) (60.2.0)
Installing collected packages: nibabel
Successfully installed nibabel-4.0.2
```

Nifti 文件格式 (*.nii, *.nii.gz)

Python读取NIFTI文件：**使用nibabel库**

读取头文件信息

- **新建 .py 文件**
  - 点击File>>New … >> Python Files,
  - 在弹出的下面对话框中输入名字nibabel_rd_niigz，不用加后缀，
  - 点击回车键



Nifti 文件格式 (*.nii, *.nii.gz)

Python读取NIFTI文件：**使用nibabel库**

读取头文件信息

- **输入代码并运行**

nibabel_rd_niigz.py

```python
import nibabel as nib
filename = 'data/patient45_ED.nii.gz'
img = nib.load(filename)
header= img.header
print(header)
```

Run nibabel_rd_niigz

```
C:\Users\20151\anaconda3\envs\pytorch_sy\python.exe D:\pythonProject\MedNN\nibabel_rd_niigz.py
<class 'nibabel.nifti1.Nifti1Header'> object, endian='<'
sizeof_hdr      : 348
data_type       : b''
db_name         : b''
extents         : 0
session_error   : 0
regular         : b'r'
dim_info        : 0
dim             : [  3 271 271 193   1   1   1   1]
intent_p1       : 0.0
intent_p2       : 0.0
intent_p3       : 0.0
intent_code     : none
datatype        : float32
bitpix          : 32
slice_start     : 0
pixdim          : [1.000e+00 5.763e-04 5.763e-04 5.763e-04 0.000e+00 0.000e+00 0.000e+00
 0.000e+00]
vox_offset      : 0.0
scl_slope       : nan
```

Nifti 文件格式 (*.nii, *.nii.gz)

Python读取NIFTI文件：**使用nibabel库**

读取图像矩阵信息

- **输入代码并运行**

nibabel_rd_niigz.py

```python
import nibabel as nib
filename = 'data/patient45_ED.nii.gz'
img = nib.load(filename)
#header= img.header
#print(header)
data = img.get_fdata()
print(data.shape)
```

Run nibabel_rd_niigz

```
C:\Users\20151\anaconda3\envs\pytorch_sy\python.exe D:\pythonProject\MedNN\nibabel_rd_niigz.py
(271, 271, 193)

Process finished with exit code 0
```

Nifti 文件格式 (*.nii, *.nii.gz)

Python读取NIFTI文件：**使用SimpleITK库**

- **安装SimpleITK库**
  - pip install SimpleITK
- **读取头文件信息**
  - 输入代码并运行

sitk_rd_niigz.py

```python
import SimpleITK as sitk
filename = 'data/patient45_ED.nii.gz'
img = sitk.ReadImage(filename)
for key in img.GetMetaDataKeys():
    print(key,img.GetMetaData(key))
```

Run sitk_rd_niigz

```
C:\Users\20151\anaconda3\envs\pytorch_sy\python.exe D:\pythonProject\MedNN\sitk_rd_niigz.py
ITK_FileNotes
ITK_original_direction [UNKNOWN_PRINT_CHARACTERISTICS]

ITK_original_spacing [UNKNOWN_PRINT_CHARACTERISTICS]

aux_file
bitpix 32
cal_max 0
cal_min 0
datatype 16
descrip
dim[0] 3
dim[1] 271
dim[2] 271
dim[3] 193
dim[4] 1
```

Nifti 文件格式 (*.nii, *.nii.gz)

Python读取NIFTI文件：**使用SimpleITK库**

读取图像矩阵信息

- **输入代码并运行**

sitk_rd_niigz.py

```python
import SimpleITK as sitk
filename = 'data/patient45_ED.nii.gz'
img = sitk.ReadImage(filename)
#for key in img.GetMetaDataKeys():
#    print(key,img.GetMetaData(key))

#读取图像矩阵
data = sitk.GetArrayFromImage(img)
#获取体素大小
spacing = img.GetSpacing()
# 获取坐标图像原点
origin = img.GetOrigin()
# 获取图像方位
direction = img.GetDirection()

print(spacing, origin, direction)
```

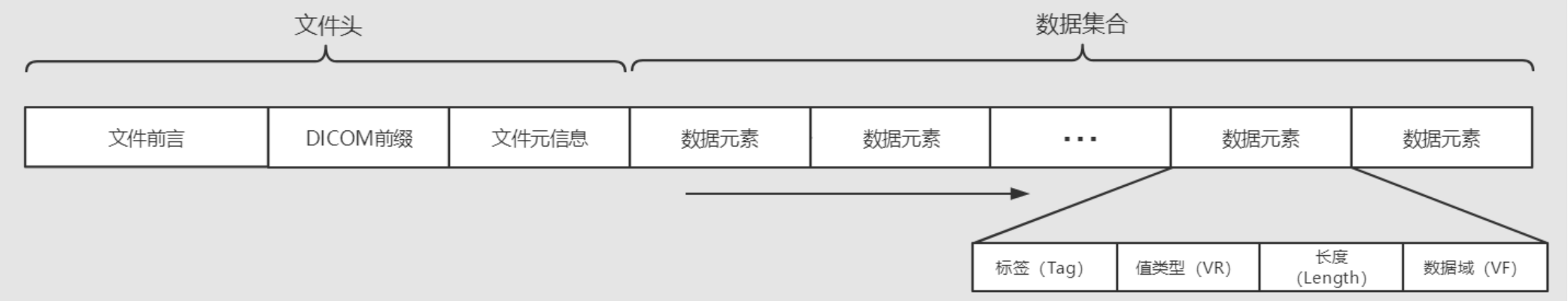
Run sitk_rd_niigz

```
C:\Users\20151\anaconda3\envs\pytorch_sy\python.exe D:\pythonProject\MedNN\sitk_rd_niigz.py
(0.0005763000226579607, 0.0005763000226579607, 0.0005763000226579607) (0.0, 0.0, 0.0) (1.0, 0.0, 0.0, 0.0, 1.0, 0.0, 0.0, 0.0, 1.0)

Process finished with exit code 0
```

Current File

Dicom 文件格式 (*.dcm)

1. 什么是DICOM?
   - DICOM(Digital Imaging and Communications in Medicine)是指医疗数字影像传输协定，是用于医学影像处理、储存、打印、传输的一组通用的标准协定。
   - 它包含了文件格式的定义以及网络通信协议。DICOM是以TCP/IP为基础的应用协定，并以TCP/IP联系各个系统。两个能接受DICOM格式的医疗仪器间，可通过DICOM格式的文件，来接收与交换影像及病人资料。
   - 目前，DICOM被广泛应用于放射医疗，心血管成像以及放射诊疗诊断设备（X射线，CT，核磁共振，超声等），并且在眼科和牙科等其它医学领域得到越来越深入广泛的应用。
2. DICOM的存储格式
   - DICOM文件的整体结构如下表所示. 首先是文件前言，128字节（没有任何信息，可以跳过）。然后是DICOM前缀4个字节。最后是数据元素，数据元素均是以key,value的形式提供，每个tag就是dicom里面定义好的字典，我们在解析时根据需要只需关注特定的tag即可，dicom全部的tag至少有 2000个。



Dicom 文件格式 (*.dcm)

3. 读取dicom格式文件

- **方法一** 使用pydicom库
  - 首先安装pydicom库

```
Terminal    Local

yapf                    0.40.2                  pypi_0    pypi
zipp                    3.15.0                  pypi_0    pypi
zlib                    1.2.13              h8cc25b3_0    defaults
zstd                    1.5.5               hd43e919_0    defaults
(pytorch_sy) PS D:\pythonProject\MedNN> pip install pydicom
Looking in indexes: https://pypi.tuna.tsinghua.edu.cn/simple, http://pypi.douban.com/simple,
stunique.com
WARNING: Retrying (Retry(total=4, connect=None, read=None, redirect=None, status=None)) afte
ple/pydicom/
WARNING: Retrying (Retry(total=3, connect=None, read=None, redirect=None, status=None)) afte
ple/pydicom/
WARNING: Retrying (Retry(total=2, connect=None, read=None, redirect=None, status=None)) afte
ple/pydicom/
WARNING: Retrying (Retry(total=1, connect=None, read=None, redirect=None, status=None)) afte
ple/pydicom/
WARNING: Retrying (Retry(total=0, connect=None, read=None, redirect=None, status=None)) afte
ple/pydicom/
Collecting pydicom
  Downloading http://mirrors.ustc.edu.cn/pypi/packages/35/2a/8c0f6fe243e6b6793868c6834203a44
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 1.8/1.8 MB 8.7 MB/s eta 0:00:00
Installing collected packages: pydicom
Successfully installed pydicom-2.4.4
```

Dicom 文件格式 (*.dcm)

3. 读取dicom格式文件

- **方法一** 使用pydicom库
  - **读取所有tag信息**

pydicom_rd_dcm.py

```python
import pydicom
dc_rd = pydicom.dcmread("data\IM-0001-0375.dcm")
print(dc_rd)
```

Run pydicom_rd_dcm

```
C:\Users\20151\anaconda3\envs\pytorch_sy\python.exe D:\pythonProject\MedNN\pydicom_rd_dcm.py
Dataset.file_meta -------------------------------
(0002, 0000) File Meta Information Group Length  UL: 188
(0002, 0001) File Meta Information Version       OB: b'\x00\x01'
(0002, 0002) Media Storage SOP Class UID         UI: CT Image Storage
(0002, 0003) Media Storage SOP Instance UID      UI: 1.2.840.113704.1.111.3352.1107859876.1643
(0002, 0010) Transfer Syntax UID                 UI: Explicit VR Little Endian
(0002, 0012) Implementation Class UID            UI: 1.3.6.1.4.1.19291.2.1
(0002, 0013) Implementation Version Name         SH: 'OSIRIX001'
(0002, 0016) Source Application Entity Title     AE: 'OSIRIX'
-------------------------------------------------
(0008, 0005) Specific Character Set              CS: 'ISO_IR 100'
(0008, 0008) Image Type                          CS: ['ORIGINAL', 'PRIMARY', 'AXIAL', 'HELIX']
(0008, 0012) Instance Creation Date              DA: '20050201'
(0008, 0013) Instance Creation Time              TM: '120000.000000'
(0008, 0016) SOP Class UID                       UI: CT Image Storage
(0008, 0018) SOP Instance UID                    UI: 1.2.840.113704.1.111.3352.1107859876.1643
(0008, 0020) Study Date                          DA: '20050201'
(0008, 0022) Acquisition Date                    DA: '20050208'
(0008, 0023) Content Date                        DA: '20050208'
(0008, 0030) Study Time                          TM: '120000.000000'
```

- 修改指定tag信息
- 添加tag信息

Dicom 文件格式 (*.dcm)

3. 读取dicom格式文件

- **方法一** 使用pydicom库
  - **读取**dicom**图像矩阵**

pydicom_rd_dcm.py

```python
import pydicom
import numpy as np
dc_rd = pydicom.dcmread("data\IM-0001-0375.dcm")
data = np.array(dc_rd.pixel_array)
print(data.shape)
```

Run pydicom_rd_dcm

```
C:\Users\20151\anaconda3\envs\pytorch_sy\python.exe D:\pythonProject\MedNN\pydicom_rd_dcm.py
(512, 512)

Process finished with exit code 0
```

Dicom 文件格式 (*.dcm)

3. 读取dicom格式文件

- **方法二** 使用SimpleITK库

读取所有tag信息

```
import SimpleITK as sitk
file_path = "Abd/1/IM000001"
file_reader = sitk.ImageFileReader()
file_reader.SetFileName(file_path)
data = file_reader.Execute()
# 使用这种方法读取Dicom的Tag Value
for key in file_reader.GetMetaDataKeys():
    print(key,file_reader.GetMetaData(key))
```

读取dicom图像矩阵

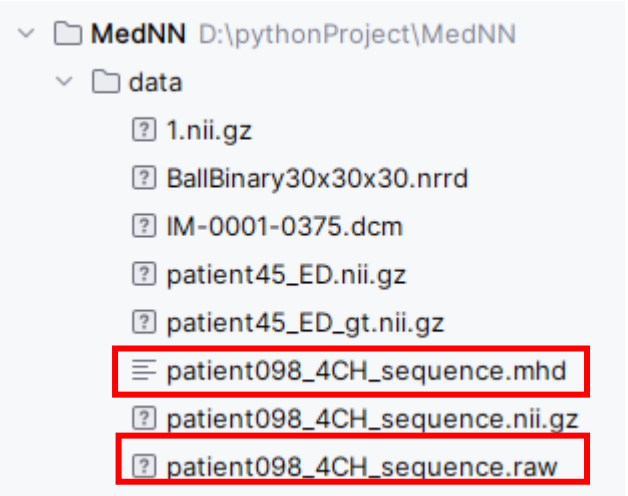
```
import SimpleITK as sitk
img = sitk.ReadImage("Abd/1/IM000001")
data=sitk.GetArrayFromImage(img)
print(data.shape)
```

读取dicom序列图像

```
import SimpleITK as sitk
folder_name = "Abd/1"#dicom序列文件夹
series_reader = sitk.ImageSeriesReader()
fileNames = series_reader.GetGDCMSeriesFileNames(folder_name)
series_reader.SetFileNames(fileNames)
images = series_reader.Execute()
data_all = sitk.GetArrayFromImage(images)#读取图像矩阵
spacing = images.GetSpacing()#获取体素大小
origin = images.GetOrigin()#坐标图像原点
direction = images.GetDirection()#图像方位
```

Meta文件格式 (*.mhd & *.raw)

1. 什么是mhd?
   - mhd是一种 metadata 格式, **.mhd用于存储头部信息，.raw用于图像数据.**
   - mhd与.raw文件必须一对一使用，**在同级目录，命名相同**
2. python读取mhd文件
   - 使用SimpleITK库，读取图像矩阵



sitk_rd_mhd.py

```python
import SimpleITK as sitk
img = sitk.ReadImage("data\patient098_4CH_sequence.mhd")
data = sitk.GetArrayFromImage(img)
print(data.shape)
```

Run sitk_rd_mhd

```
C:\Users\20151\anaconda3\envs\pytorch_sy\python.exe D:\pythonProject\MedNN\sitk_rd_mhd.py
(54, 1103, 669)

Process finished with exit code 0
```

其他格式

- nrrd格式：扩展名为 .nrrd
- Analyze格式:扩展名为 .hdr
- 2D图像 PNG, JPG,……
- ……

pip install pyprrd

```
(pytorch_sy) D:\0aMed_NN>pip install pynrrd
Looking in indexes: https://pypi.tuna.tsinghua.edu.cn/simple, http://pypi.douban.com/simple, http://mirrors.aliyun.com/pypi/simple, https://pypi.tuna.tsinghua.edu.
com
WARNING: Retrying (Retry(total=4, connect=None, read=None, redirect=None, status=None)) after connection broken by 'ProtocolError('Connection aborted.', RemoteDis
d/
WARNING: Retrying (Retry(total=3, connect=None, read=None, redirect=None, status=None)) after connection broken by 'ProtocolError('Connection aborted.', RemoteDis
d/
WARNING: Retrying (Retry(total=2, connect=None, read=None, redirect=None, status=None)) after connection broken by 'ProtocolError('Connection aborted.', RemoteDis
d/
WARNING: Retrying (Retry(total=1, connect=None, read=None, redirect=None, status=None)) after connection broken by 'ProtocolError('Connection aborted.', RemoteDis
d/
WARNING: Retrying (Retry(total=0, connect=None, read=None, redirect=None, status=None)) after connection broken by 'ProtocolError('Connection aborted.', RemoteDis
d/
Collecting pynrrd
  Downloading http://mirrors.ustc.edu.cn/pypi/packages/21/84/34fa17e364f8332021d4b0f75beecd4b95a76a41449e96ed0ddac5a767e0/pynrrd-1.1.3-py3-none-any.whl (23 kB)
Requirement already satisfied: numpy>=1.21 in c:\users\20151\anaconda3\envs\pytorch_sy\lib\site-packages (from pynrrd) (1.21.5)
Requirement already satisfied: typing_extensions in c:\users\20151\anaconda3\envs\pytorch_sy\lib\site-packages (from pynrrd) (4.4.0)
Installing collected packages: pynrrd
Successfully installed pynrrd-1.1.3
```

nrrd_rd.py

```python
import nrrd
filename = 'data/BallBinary30x30x30.nrrd'
data,header = nrrd.read(filename)
print(data.shape)
print(header)
```

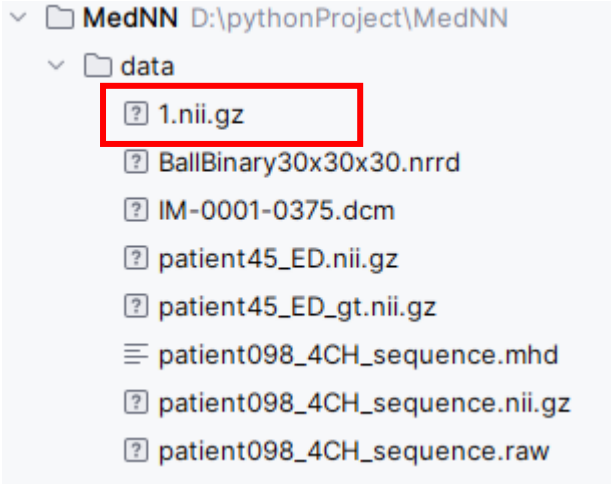
```
C:\Users\20151\anaconda3\envs\pytorch_sy\python.exe D:\0aMed_NN\nrrd_rd.py
(30, 30, 30)
OrderedDict([('type', 'short'), ('dimension', 3), ('space', 'left-posterior-superior'), ('sizes', array([30, 30, 30])), ('space directions', array([[1., 0., 0.],
       [0., 1., 0.],
       [0., 0., 1.]])), ('kinds', ['domain', 'domain', 'domain']), ('endian', 'little'), ('encoding', 'raw'), ('space origin', array([0., 0., 0.]))])

Process finished with exit code 0
```

格式转换

- **Dicom格式转变为NIFTI格式**

dcm_to_nifti.py

```python
import SimpleITK as sitk
folder_name = 'data'
series_reader = sitk.ImageSeriesReader()
fileNames = series_reader.GetGDCMSeriesFileNames(folder_name)
series_reader.SetFileNames(fileNames)
images = series_reader.Execute()
sitk.WriteImage(images,'data/1.nii.gz')
```

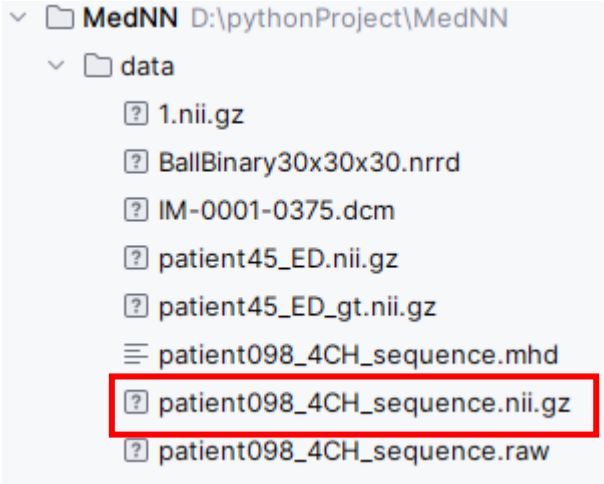


格式转换

- **mhd 转NIFTI格式**

mhd_to_nifti.py

```python
import SimpleITK as sitk
img = sitk.ReadImage("data\patient098_4CH_sequence.mhd")
sitk.WriteImage(img,'data/patient098_4CH_sequence.nii.gz')
```

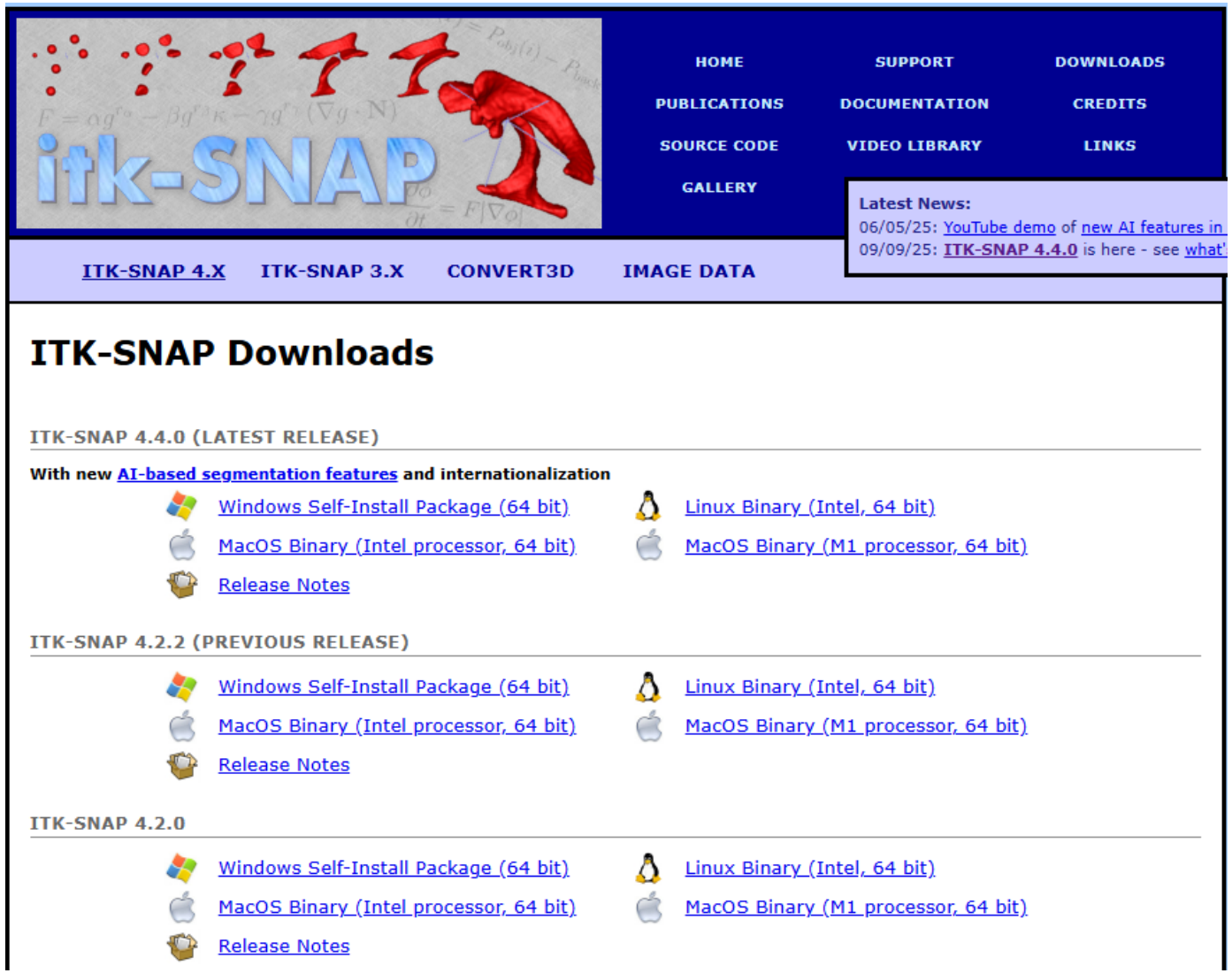


ITK-SNAP

- ITK-SNAP是一款**免费、开源**的多平台医学图像分割软件，主要用于3D和4D生物医学图像的手动和半自动分割。软件概述。
- 软件**功能集中于图像分割任务**，用户能够快速上手。
- 核心功能
  - 图像分割：
    - 支持手动描绘、半自动分割（如基于主动轮廓方法）以及交互式分割。
  - 图像可视化：
    - 可加载多种医学图像格式（如MRI、CT），以三维方式查看图像结构，帮助理解解剖信息。
  - 自动分割：
    - 利用已有标注数据训练模型，实现新图像的自动识别和分割。
  - 辅助分析：
    - 可用于肿瘤边界识别、体积测量及解剖结构研究，为临床诊断提供支持。
  - 多平台支持：
    - Windows、Mac和Linux系统均可使用，且为开源软件。

ITK-SNAP

- 下载：ITK-SNAP Home
- 安装：
- 注：安装路径无中文



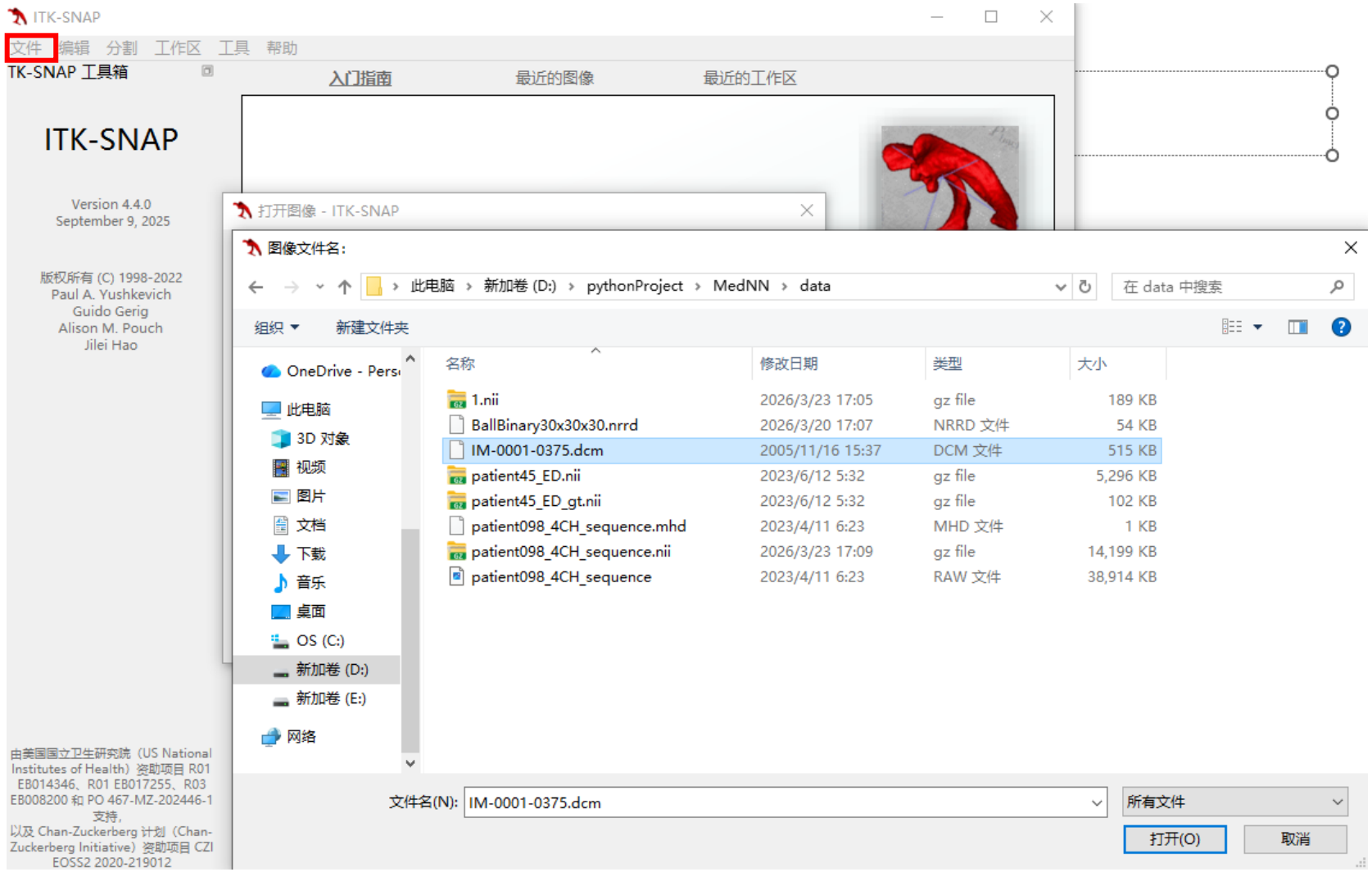
开始使用ITK-SNAP

安装完成显示界面

开始使用ITK-SNAP

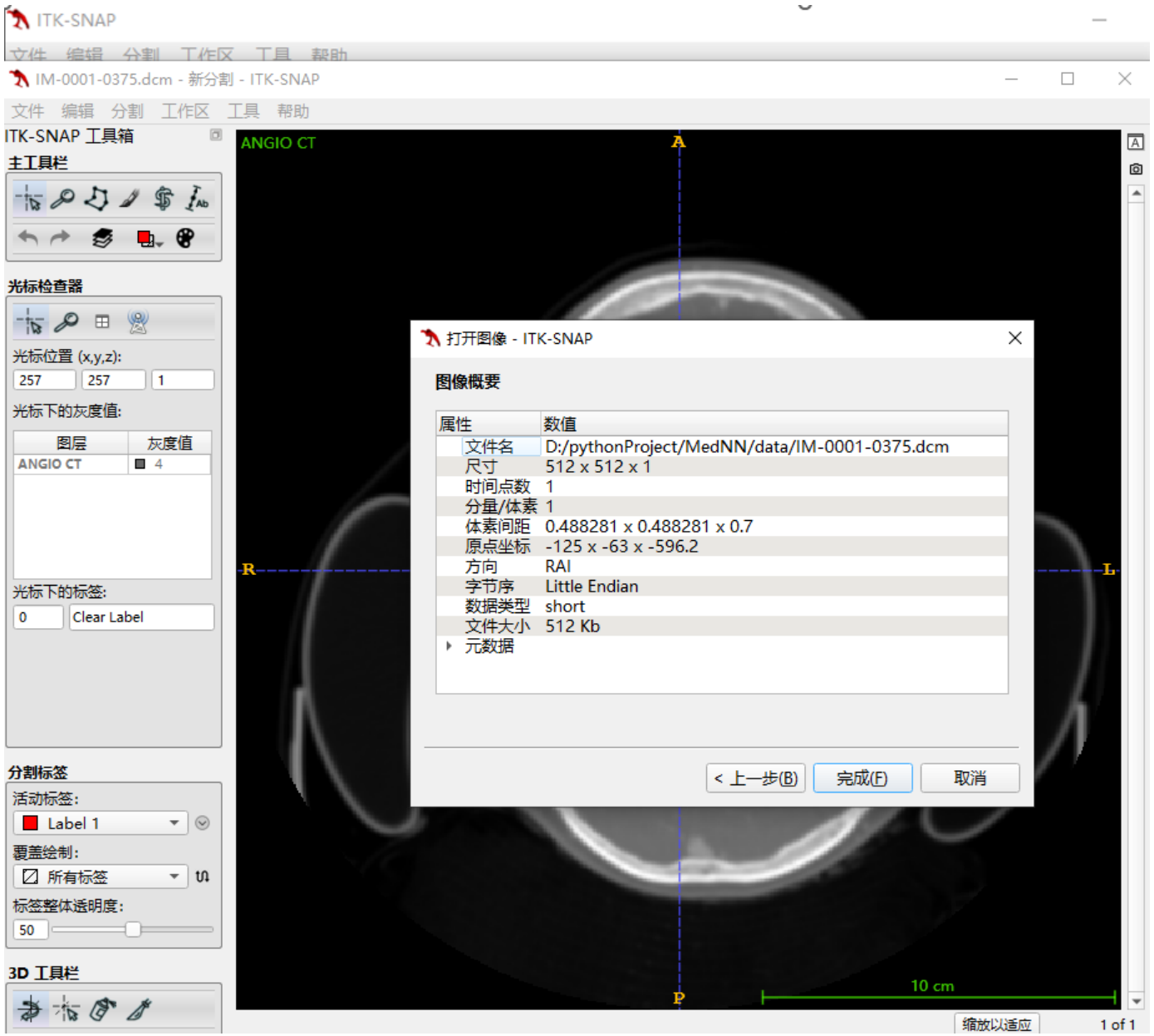
ITK-Snap支持查看 DICOM,NIFTI,mhd等格式的图像。可通过菜单栏导入或者鼠标拖入。

打开数据：菜单栏：文件>> 打开主图像，弹出如下对话框



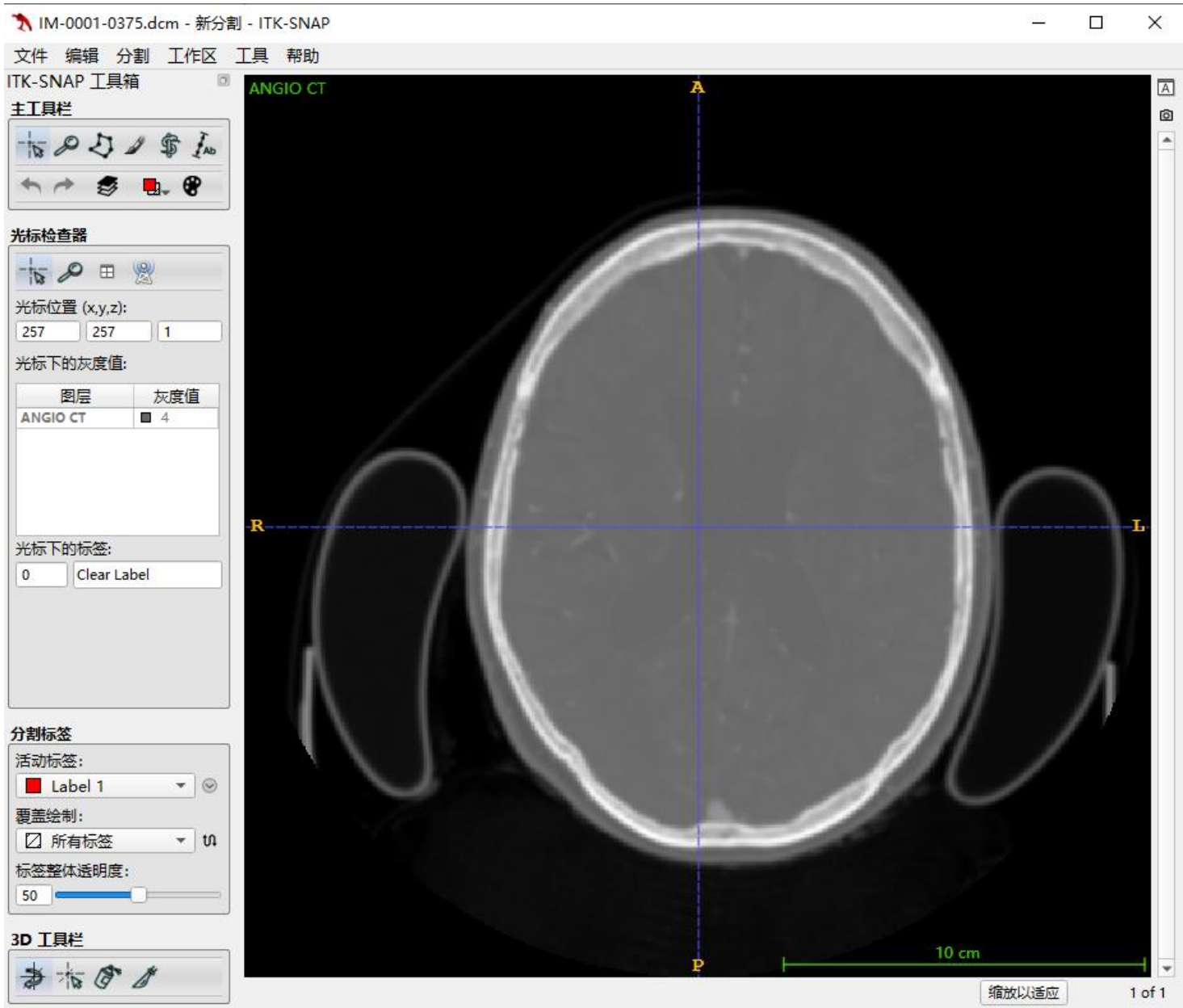
开始使用ITK-SNAP

点击完成后，首先显示的是图像概要。



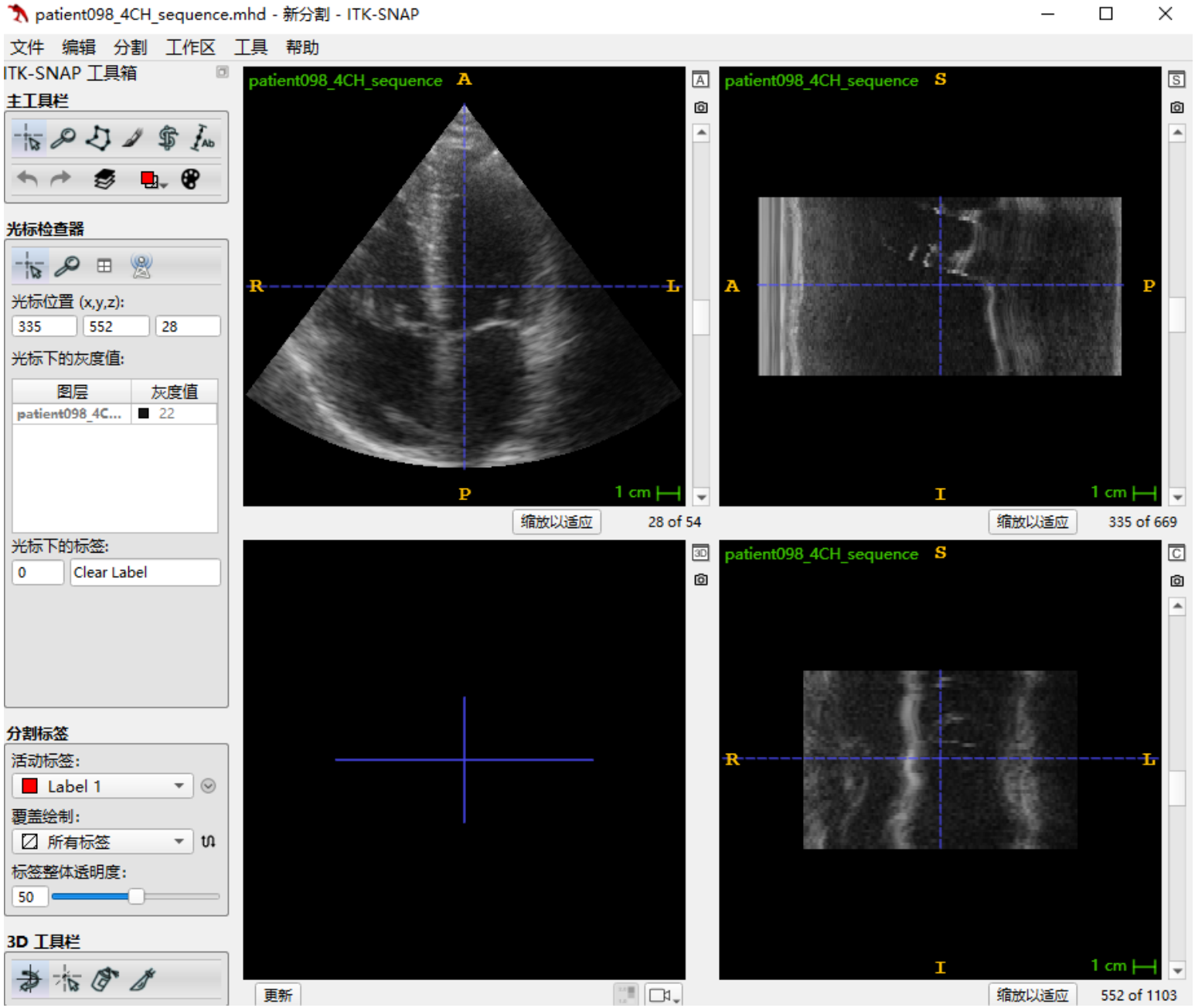
开始使用ITK-SNAP

- 打开的dcm格式文件
- 数据标注



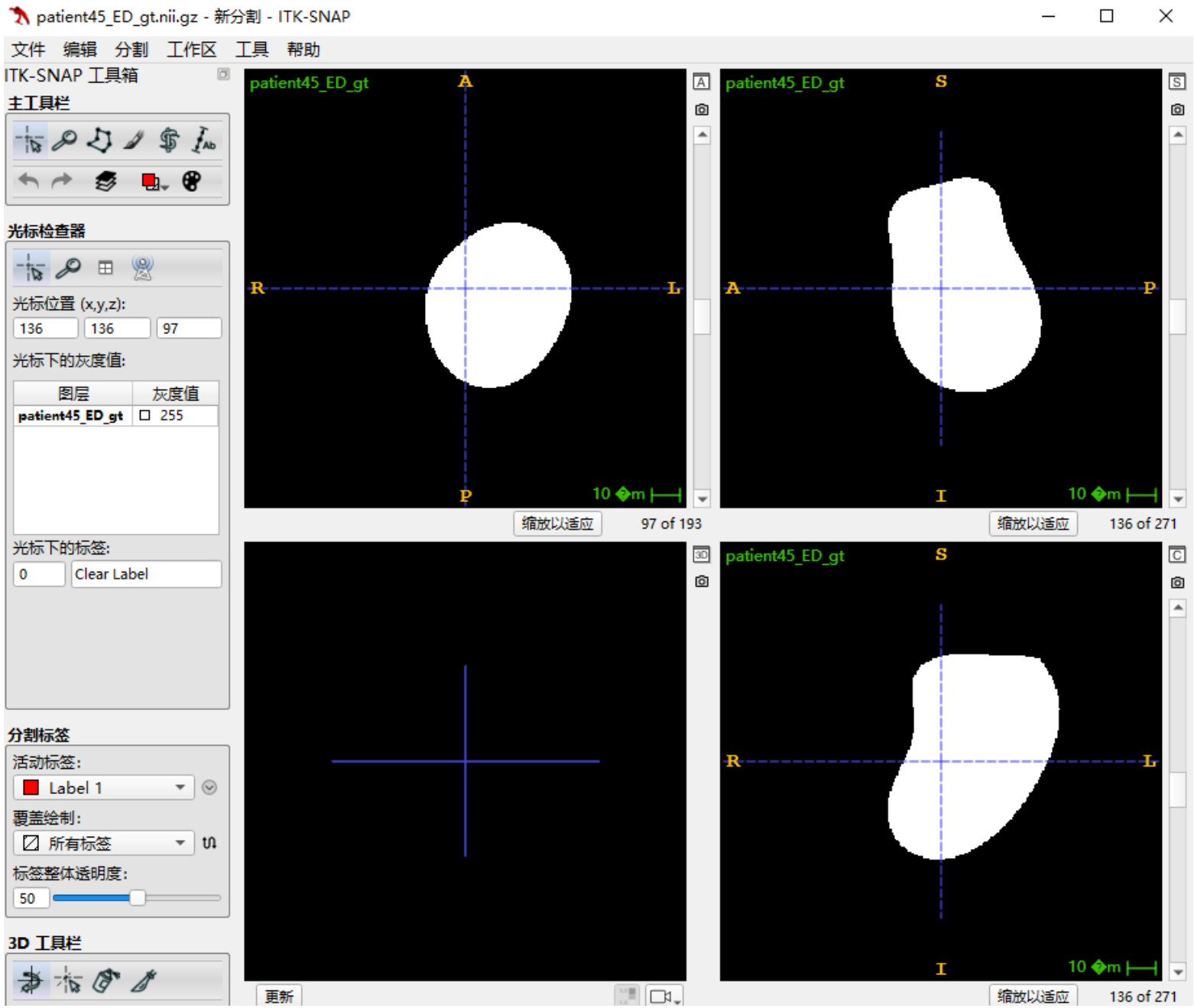
开始使用ITK-SNAP

打开的mhd格式文件



开始使用ITK-SNAP

打开的NIFTI格式文件



- siyusa/MedNN （https://github.com/siyusa/MedNN）
- 使用jupyter notebook
  - 安装库
  - 加载各格式的数据
  - 进行数据格式转换